

Investigating the Impact of Class Imbalance Handling on Machine Learning Performance and SHAP-Based Interpretability for Predictive Maintenance

Syed Yousaf Shah

Email: syedyousafshah997@gmail.com

Abstract—Predictive maintenance (PdM) systems use machine-learning models to identify equipment failures before they interrupt production. However, failure observations are typically rare, making class imbalance a central challenge in failure detection. This study investigates whether the choice of imbalance-handling strategy affects both predictive performance and the explanations produced by an interpretable machine-learning model. Experiments were conducted on the AI4I 2020 Predictive Maintenance Dataset using Random Forest, support vector machine (SVM), and XGBoost classifiers under three conditions: no imbalance handling, class weighting, and synthetic minority over-sampling technique (SMOTE). An 80/20 stratified train-test split was used, with the test set kept untouched by resampling. Accuracy, precision, recall, F1-score, ROC-AUC, and precision-recall area under the curve (PR-AUC) were evaluated, with PR-AUC emphasized for model selection. Among the nine configurations, XGBoost with SMOTE achieved the highest PR-AUC of 0.836 and a recall of 0.794. SHAP analysis was then used to examine global and local model behavior and to compare feature-importance patterns between XGBoost models trained with class weighting and SMOTE. The two approaches produced highly similar but non-identical importance rankings, indicating that imbalance handling can influence not only predictive performance but also the interpretation of model behavior.

Index Terms—Predictive maintenance, class imbalance, SMOTE, XGBoost, Random Forest, SVM, SHAP, explainable artificial intelligence.

I. INTRODUCTION

Industrial systems increasingly rely on sensors and automated monitoring to observe equipment operating conditions. Predictive maintenance uses these data to estimate the likelihood of failure before a breakdown occurs, enabling maintenance to be scheduled proactively rather than after an unexpected interruption. Machine-learning methods are attractive for this task because they can model nonlinear relationships among variables such as torque, rotational speed, temperature, and tool wear [1], [2].

A major difficulty in predictive-maintenance classification is that failure events are much less common than normal operating observations. The AI4I 2020 dataset contains 10,000 observations and only 339 machine-failure cases, corresponding to a failure rate of 3.39% [3]. Under such a distribution, a classifier can achieve high accuracy by favoring the majority class while still missing a substantial proportion of failures.

For maintenance applications, this behavior is undesirable because a false negative represents a failure that the system failed to identify.

Class imbalance can be addressed at either the algorithmic or data level. Class weighting changes the relative penalty assigned to classes during model training, while SMOTE creates synthetic minority observations using neighboring minority-class samples [4]. These approaches are not equivalent: class weighting changes the learning objective without changing the training observations, whereas SMOTE changes the effective training distribution. Consequently, their influence may extend beyond conventional performance metrics.

Explainability provides a second dimension to this problem. SHAP (SHapley Additive exPlanations) assigns feature-level contributions to individual predictions and can be aggregated to obtain global feature-importance patterns [5]. In a predictive-maintenance setting, such explanations can help analysts understand which operating variables are associated with predicted failures. However, if the training strategy changes the learned decision function, the resulting SHAP explanations may also change. This creates an important methodological question: should a practitioner consider imbalance handling only as a performance optimization, or also as a factor that can affect the model's explanatory narrative?

This study therefore asks: How do no handling, class weighting, and SMOTE affect the predictive performance of Random Forest, SVM, and XGBoost models for machine-failure detection, and how does imbalance handling influence SHAP-based explanations of the selected model?

The study makes four practical contributions. First, it establishes a leakage-aware binary classification pipeline for the AI4I 2020 dataset. Second, it evaluates nine combinations of model family and imbalance strategy using metrics suited to rare-event detection. Third, it examines global and local SHAP explanations for the selected model. Fourth, it directly compares SHAP feature-importance patterns between class-weighted and SMOTE versions of the same XGBoost model. The contribution is empirical rather than the invention of a new classifier or sampling algorithm.

II. RELATED WORK

Five studies were selected as the core literature for this investigation. They were chosen because they address the AI4I 2020 dataset, predictive-maintenance classification, ensemble learning, imbalance, or explainability.

A. Matzka: *Explainable Predictive Maintenance*

Matzka introduced the AI4I 2020 dataset as a realistic synthetic predictive-maintenance benchmark and investigated explainability using an explainable model and an explanatory interface [1]. The work is foundational for the present study because it establishes the dataset and emphasizes that predictive-maintenance systems should provide understandable explanations. However, the focus was not a controlled comparison of class-imbalance strategies and their effect on SHAP explanations.

B. Kanar et al.: *Ensemble Learning*

Kanar, Çiftınar, and Erzurum Cicek evaluated ensemble approaches for failure prediction using the AI4I 2020 dataset and reported approximately 99% accuracy for their ensemble models [6]. The study demonstrates that combining classifiers can improve predictive performance, but its emphasis is primarily performance-oriented. For a strongly imbalanced dataset, accuracy alone does not reveal the complete minority-class behavior, leaving room for evaluation using PR-AUC, recall, and F1-score.

C. Rismawan and Nirmala: *Leakage-Free Imbalance Evaluation*

Rismawan and Nirmala provide a particularly relevant recent benchmark by evaluating five algorithms and four imbalance-handling strategies under a leakage-free factorial design on the AI4I 2020 dataset [7]. Their results show that synthetic oversampling is not universally beneficial and that the effect of imbalance handling depends on the model and task. This finding motivates the present controlled comparison, while the present study adds a specific focus on the relationship between imbalance handling and SHAP explanations.

D. Ghasemkhani et al.: *Balanced K-Star*

Ghasemkhani, Aktas, and Birant proposed Balanced K-Star, an explainable machine-learning approach for IoT-enabled predictive maintenance [8]. The work demonstrates that imbalance-aware predictive maintenance can be designed together with explainability. Nevertheless, its focus is a specialized classifier rather than a factorial comparison of common models under alternative imbalance strategies.

E. Zhao et al.: *ML and SHAP for Milling-Machine Maintenance*

Zhao et al. investigated predictive maintenance on the AI4I 2020 dataset using multiple machine-learning models and SHAP analysis [9]. The study reported strong performance from ensemble methods such as XGBoost and Random Forest and used SHAP to identify influential variables. The present

work differs by making imbalance handling an explicit experimental factor and by comparing explanations across two imbalance treatments for the same model family.

Taken together, the reviewed literature demonstrates three established directions: predictive-maintenance classification, imbalance handling, and explainable machine learning. The specific intersection studied here—whether the choice between class weighting and SMOTE changes the feature-importance pattern of the same predictive model—receives less direct attention in the reviewed studies. This gap motivates the experimental design used in this paper.

TABLE I
COMPARISON OF THE FIVE CORE REVIEWED STUDIES

Study	Dataset/Task	Main Focus
Matzka [1]	AI4I 2020	XAI/PdM benchmark
Kanar et al. [6]	AI4I 2020	Ensemble failure prediction
Rismawan & Nirmala [7]	AI4I 2020	Factorial imbalance evaluation
Ghasemkhani et al. [8]	PdM/IoT	Explainable imbalance-aware model
Zhao et al. [9]	AI4I 2020	ML + SHAP

III. DATASET

The AI4I 2020 Predictive Maintenance Dataset is a synthetic multivariate dataset published through the UCI Machine Learning Repository [3]. It contains 10,000 observations designed to reflect predictive-maintenance data encountered in industrial settings. The UCI description identifies Type as a categorical product-quality variant and provides air temperature, process temperature, rotational speed, torque, and tool wear as operational features. The target Machine failure indicates whether a failure occurred in an observation.

The dataset also contains five failure-mode indicators: tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF), and random failure (RNF). These variables are target/failure information rather than ordinary sensor predictors and were excluded from the feature matrix in the present binary prediction experiment to avoid using information that would not be appropriate as a live predictor.

The class distribution is highly skewed: 9,661 observations represent normal operation and 339 represent machine failure. This 3.39% failure rate creates an approximately 28.5:1 majority-to-minority ratio and motivates the use of imbalance-aware evaluation and training strategies.

IV. METHODOLOGY

A. Data Preprocessing

Columns that identify individual records or products rather than operational behavior were excluded from the predictor matrix. The categorical Type variable was one-hot encoded.

TABLE II
PRINCIPAL DATASET VARIABLES

Variable	Role	Description
Type	Feature	Product quality variant: L, M, or H
Air temperature [K]	Feature	Ambient air temperature
Process temperature [K]	Feature	Process operating temperature
Rotational speed [rpm]	Feature	Machine rotational speed
Torque [Nm]	Feature	Applied torque
Tool wear [min]	Feature	Accumulated tool wear
Machine failure	Target	Binary failure label
TWF/HDF/PWF/OSF/RNF	Failure labels	Specific failure-mode indicators

Numeric variables were standardized using StandardScaler in the implemented pipeline. Although scaling is particularly important for the RBF-SVM because distances and kernel calculations depend on feature magnitudes, applying the same standardized representation to the tree-based models does not fundamentally change their split-based mechanism.

Two engineered variables were created during exploratory analysis. Temperature difference was calculated as the difference between process and air temperature, and mechanical power was calculated from torque and rotational speed using $P = \tau\omega$, with angular speed converted from revolutions per minute. These variables provide physically interpretable combinations of the original measurements and can help the models represent operating conditions in a more direct form.

The data were divided using an 80/20 stratified train-test split with a fixed random state. Stratification preserves the approximate failure proportion in both partitions. The test set was kept completely untouched by SMOTE and was used only for final evaluation. This separation is essential because resampling before the split can allow information from synthetic or duplicated minority observations to influence evaluation and produce optimistic estimates.

B. Imbalance-Handling Strategies

Three training conditions were compared. The first is No Handling, which provides the baseline using the original class distribution. The second is Class Weighting, which increases the effective importance of minority-class errors during optimization. For Random Forest and SVM, balanced class weights were used; for XGBoost, the positive-class weight was set using the ratio of negative to positive training examples. The third condition is SMOTE, which synthesizes minority-class observations from neighboring minority samples [4].

SMOTE was applied only to the training data after the preprocessing transformation and was integrated through an imbalanced-learn pipeline. This design prevents the test set from being altered and reduces the risk of leakage during training.

C. Models

Random Forest was selected as a bagging-based ensemble of decision trees. Its ability to model nonlinear relationships and interactions makes it suitable for tabular sensor data. SVM with an RBF kernel was included as a fundamentally different decision-boundary approach and provides a useful test of whether imbalance strategies behave similarly for a distance/kernel-based model. XGBoost was selected as a gradient-boosted tree method known for strong performance on structured data [10].

TABLE III
EVALUATED MACHINE-LEARNING MODELS

Model	Learning Principle	Reason for Inclusion
Random Forest	Bagged decision trees	Robust non-linear base-line
RBF-SVM	Kernel-based margin classifier	Contrasting non-tree model
XGBoost	Gradient-boosted trees	Strong tabular-data learner

D. Experimental Factorial Design

The experiment evaluates 3 model families $\times$ 3 imbalance conditions, resulting in nine configurations. All nine configurations use the same held-out test set. This design makes it possible to distinguish model effects from imbalance-handling effects and to determine whether a strategy is consistently useful across algorithms.

V. EXPERIMENTAL SETUP

A. Evaluation Metrics

Accuracy was reported for completeness but was not treated as the primary decision criterion because the failure class is rare. Precision measures the proportion of predicted failures that are actual failures, while recall measures the proportion of actual failures that are detected. F1-score summarizes the precision-recall trade-off. ROC-AUC measures class discrimination over thresholds, whereas PR-AUC focuses directly on the precision-recall relationship and is especially informative for a rare positive class.

PR-AUC was used as the primary metric for model selection, with recall and F1-score considered as complementary indicators. This reflects the operational importance of detecting failures while limiting excessive false alarms.

B. Reproducibility

The experimental pipeline uses a fixed random state for the train-test split. Preprocessing transformations are fitted using training data, and the held-out test data are transformed using those fitted operations. SMOTE is restricted to training data. The reported results should therefore be reproducible provided

the same dataset version, software environment, preprocessing choices, random state, and model hyperparameters are used.

Software used in the implementation includes Python-based scientific-computing and machine-learning libraries, including pandas, NumPy, scikit-learn, imbalanced-learn, XGBoost, and SHAP. Exact package versions should be inserted before final submission if they are not already recorded in the project environment.

VI. RESULTS

A. Predictive Performance

XGBoost with SMOTE achieved the highest PR-AUC of 0.836 and the highest recall among the nine configurations at 0.794. Its precision was 0.740 and its F1-score was 0.766. XGBoost without handling reached a slightly lower PR-AUC of 0.828 while retaining higher precision (0.875), illustrating that maximizing recall and PR-AUC can involve additional false positives.

Random Forest also showed a strong baseline. Its no-handling configuration produced PR-AUC of 0.807 and precision of 0.929, whereas SMOTE increased recall from 0.574 to 0.750 but reduced precision from 0.929 to 0.515. The SVM displayed an even stronger recall response to imbalance handling: class weighting increased recall from 0.191 to 0.897, but precision fell to 0.272. This demonstrates that an increase in recall does not automatically mean that a configuration is preferable for deployment.

B. Confusion-Matrix and Error Analysis

For the selected XGBoost + SMOTE configuration, the test set contains 68 actual failures. The classification results report 54 correctly detected failures and 14 missed failures, giving recall of approximately 0.794. There were also 19 false positives, consistent with the reported precision of approximately 0.74. Thus, the model reduces missed failures at the cost of generating additional false alarms.

The 14 false negatives are particularly important because they represent failures that were not detected. The supplied experimental analysis indicates that TWF and HDF were prominent among these missed failures. Such error analysis should be interpreted as a diagnostic analysis rather than proof that these failure modes are intrinsically harder to predict, because the dataset is synthetic and the observed counts depend on the trained model and selected test split.

C. SHAP Global Interpretability

SHAP was applied to the selected model to quantify feature contributions to predictions. The global bar plot identifies Torque and Tool wear as the most influential variables in the reported experiment. The SHAP beeswarm visualization further indicates that high values of torque, tool wear, and temperature-related variables can push model outputs toward the failure class in the examined predictions.

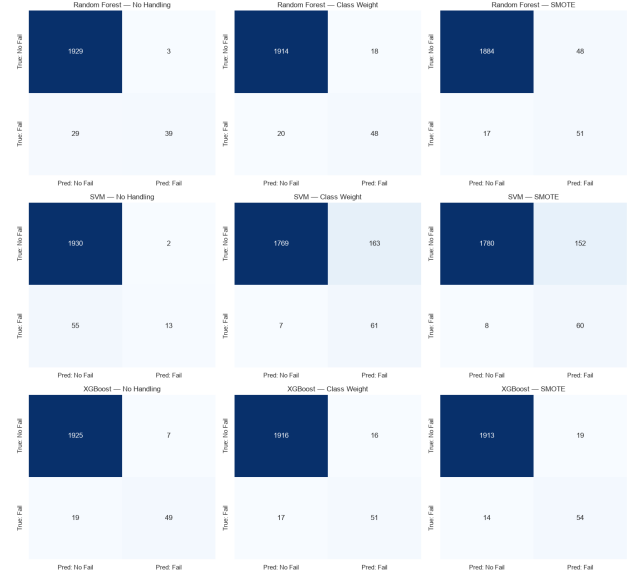


Fig. 1. Confusion matrices for Random Forest, SVM, and XGBoost under the three imbalance strategies.

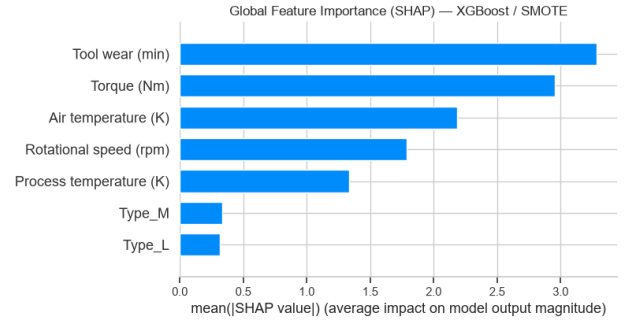


Fig. 2. Global SHAP feature importance for the selected XGBoost + SMOTE model.

D. SHAP Local Interpretability

Local SHAP explanations were examined for representative true-positive and false-negative cases. For the reported true-positive case, torque and rotational speed were among the strongest contributors toward the predicted failure. The false-negative example showed comparatively borderline feature behavior, suggesting that the case was less clearly separated from the normal class by the learned decision function.

E. Effect of Imbalance Handling on SHAP Importance

The two XGBoost configurations retained the same two leading features, Torque and Tool wear, but the relative ordering of several lower-ranked features changed. The reported correlation between the two feature-importance vectors was 0.949, indicating strong overall agreement but not complete equivalence. This result supports a cautious conclusion: the imbalance strategy did not fundamentally replace the dominant feature drivers in this experiment, but it did modify the relative contribution assigned to secondary features.

TABLE IV
TEST-SET PERFORMANCE OF THE NINE CONFIGURATIONS

Model	Strategy	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
RF	No handling	0.984	0.929	0.574	0.709	0.960	0.807
RF	Class weight	0.981	0.727	0.706	0.716	0.970	0.782
RF	SMOTE	0.968	0.515	0.750	0.611	0.966	0.736
SVM	No handling	0.972	0.867	0.191	0.313	0.946	0.642
SVM	Class weight	0.915	0.272	0.897	0.418	0.966	0.616
SVM	SMOTE	0.920	0.283	0.882	0.429	0.965	0.623
XGB	No handling	0.987	0.875	0.721	0.790	0.976	0.828
XGB	Class weight	0.984	0.761	0.750	0.756	0.970	0.828
XGB	SMOTE	0.984	0.740	0.794	0.766	0.970	0.836

TABLE V
CLASSIFICATION REPORT FOR XGBOOST + SMOTE

Class	Precision	Recall	F1	Support
No Failure	0.99	0.99	0.99	1932
Failure	0.74	0.79	0.77	68
Accuracy	—	—	0.98	2000
Macro Avg.	0.87	0.89	0.88	2000

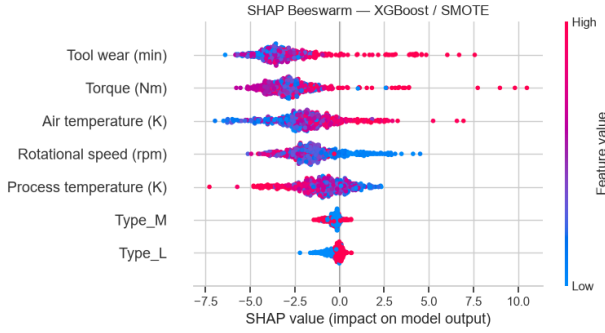


Fig. 3. SHAP beeswarm plot showing feature contribution magnitude and direction.

VII. DISCUSSION

A. Model Performance

The experiments show that model family and imbalance strategy interact. XGBoost produced the strongest overall minority-class performance, while SVM was highly sensitive to class weighting and SMOTE. Random Forest provided a competitive baseline but exhibited a pronounced precision-recall trade-off when SMOTE was applied. These results are consistent with the broader observation that no single imbalance strategy is guaranteed to improve every classifier [7].

B. Performance Versus Operational Cost

The selected XGBoost + SMOTE model was chosen because it achieved the highest PR-AUC and a strong recall. However, its precision was lower than XGBoost without handling. In practical maintenance systems, the preferred operating point should depend on the relative costs of missed failures and unnecessary maintenance alerts. Therefore, the

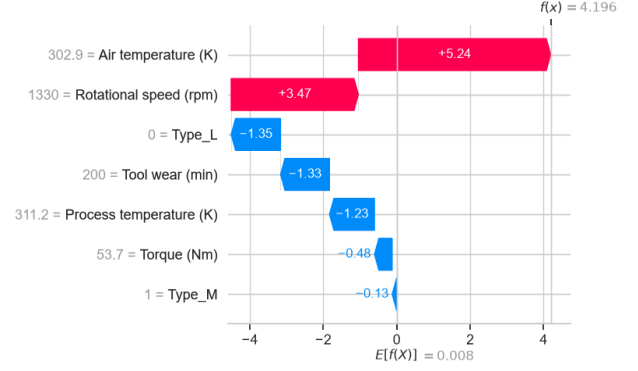


Fig. 4. Local SHAP explanation for a correctly detected failure.

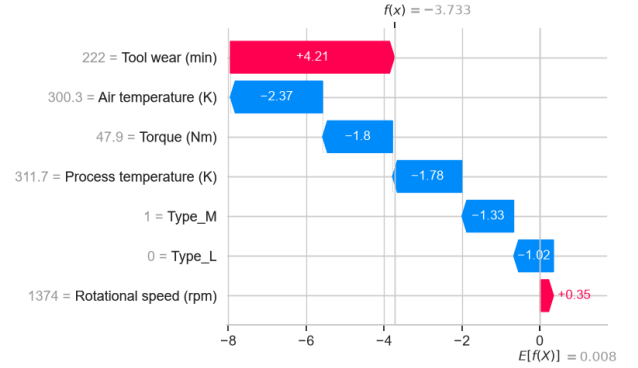


Fig. 5. Local SHAP explanation for a missed failure.

model selected by this study should not be interpreted as universally optimal for every industrial cost structure.

C. What SHAP Adds

Performance metrics describe what the model achieves, whereas SHAP provides evidence about how the model arrives at individual outputs. The comparison between class weighting and SMOTE is important because both methods can yield similar predictive scores while producing different feature contributions. In this experiment, the dominant features remained stable, but secondary rankings changed. This suggests that explainability should be considered part of the experimental

TABLE VI
SHAP FEATURE-IMPORTANCE COMPARISON FOR XGBOOST

Feature	Class Weight	SMOTE	Rank Change
Torque [Nm]	4.2651	2.9555	+1
Tool wear [min]	3.3715	3.2820	-1
Rotational speed [rpm]	2.0548	1.7854	+1
Air temperature [K]	2.0534	2.1871	-1
Process temperature [K]	1.0550	1.3315	0
Type_L	0.4964	0.3136	+1
Type_M	0.1976	0.3343	-1

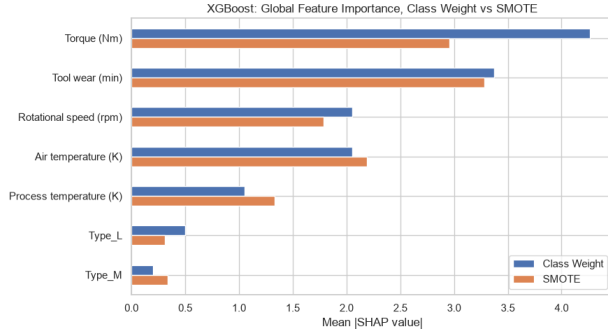


Fig. 6. Comparison of SHAP feature importance between class-weighted and SMOTE XGBoost models.

design rather than an afterthought added only after a model has been selected.

D. Failure Cases and Practical Implications

False negatives are particularly important in predictive maintenance because they correspond to missed opportunities for intervention. The selected model still missed 14 of 68 failures in the test set. Consequently, even the best configuration in this experiment should not be interpreted as a fully autonomous maintenance decision-maker. A practical deployment would require threshold tuning, cost-sensitive evaluation, additional validation data, and human or engineering oversight.

VIII. LIMITATIONS

First, AI4I 2020 is a synthetic dataset designed to reflect realistic predictive-maintenance data; it should therefore be treated as a benchmark rather than direct evidence of performance on a particular production line [3]. Real industrial data can contain sensor drift, missing measurements, changing operating regimes, maintenance interventions, and temporal dependencies that are not captured by a simple random split.

Second, the experiments use one fixed train-test split and a single set of reported experimental settings. Repeated stratified cross-validation or multiple random seeds would provide stronger evidence about the stability of the rankings. Third, only three model families were evaluated. Other algorithms may respond differently to class imbalance and explainability analysis.

Fourth, SHAP values explain model behavior; they do not establish physical causality. A feature with a large SHAP contribution should therefore be interpreted as influential to the learned model, not automatically as the physical cause

of a failure. Finally, the comparison of SHAP rankings is descriptive. Future work should quantify explanation stability statistically across repeated resampling and model fits.

IX. CONCLUSION AND FUTURE WORK

This study investigated the effect of class-imbalance handling on both predictive performance and SHAP-based interpretability for machine-failure prediction. Using the AI4I 2020 dataset, nine configurations combining Random Forest, SVM, and XGBoost with no handling, class weighting, or SMOTE were evaluated on a held-out stratified test set. XGBoost with SMOTE achieved the highest PR-AUC of 0.836 and a recall of 0.794. The results also demonstrated that improving minority-class detection can involve a precision trade-off, emphasizing the importance of selecting metrics that reflect the intended maintenance objective.

The explainability analysis provides the main research insight. Torque and tool wear remained the leading SHAP features for the evaluated XGBoost configurations, while several secondary feature rankings changed between class weighting and SMOTE. The strong but imperfect correlation of 0.949 indicates that the imbalance strategy can subtly alter the explanatory profile of a model. Accordingly, imbalance handling should be treated not only as a predictive-performance decision but also as a factor that may influence model interpretation.

Future work should validate the findings across repeated cross-validation and multiple random seeds, evaluate additional imbalance strategies, and test the methodology on real industrial datasets. Further research could also quantify explanation stability, evaluate threshold optimization under explicit maintenance costs, and extend the binary task to individual failure modes such as TWF, HDF, PWF, OSF, and RNF.

REFERENCES

- [1] S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," in *Proc. 2020 Third Int. Conf. Artificial Intelligence for Industries (AI4I)*, 2020, pp. 69–74, doi: 10.1109/AI4I49448.2020.00023.
- [2] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Lima, E. S. da Trindade, and G. P. Li, "Predictive maintenance in the Industry 4.0: A systematic literature review," *Comput. Ind. Eng.*, vol. 150, p. 106889, 2020.
- [3] UCI Machine Learning Repository, "AI4I 2020 Predictive Maintenance Dataset," Dataset 601, 2020. [Online]. Available: <https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset>
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.
- [5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.* 30, 2017, pp. 4765–4774.
- [6] P. Kanar, A. B. Çiftçin, and Z. I. Erzurum Cicek, "Ensemble learning based machine learning models for predictive maintenance," in *Proc. 5th Int. Conf. Sustainable Science and Technology (ICSuSaT)*, 2022.
- [7] T. Rismawan and I. Nirmala, "Imbalance Handling Strategies for Predictive Maintenance Under Leakage-Free Factorial Evaluation," *J. Fuzzy Syst. Control*, vol. 4, no. 1, pp. 90–102, 2026, doi: 10.59247/jfsc.v4i1.399.
- [8] B. Ghasemkhani, O. Aktas, and D. Birant, "Balanced K-Star: An Explainable Machine Learning Method for Internet-of-Things-Enabled Predictive Maintenance in Manufacturing," *Machines*, vol. 11, no. 3, p. 322, 2023, doi: 10.3390/machines11030322.

- [9] W. Zhao, J. Ding, X. Huang, and Y. Zhang, "Research on Milling Machine Predictive Maintenance Based on Machine Learning and SHAP Analysis in Intelligent Manufacturing Environment," *arXiv:2512.01205*, 2025.
- [10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [12] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.